

PopMatch

AI/ML Interview Preparation Guide

Architecture | RAG | Embeddings | ML Training | AI Agents | Python Backend

1. Project Overview - The 30-Second Pitch

PopMatch is a cross-platform mobile app (Flutter/Dart) that uses a 7-service hybrid AI recommendation engine to surface personalized movies and TV shows through a Tinder-style swipe interface. Users swipe right (like), left (dislike), up (match + watchlist), or down (skip). Every interaction feeds seven concurrent ML systems that continuously refine future recommendations.

Tech Stack:

- Frontend: Flutter/Dart (iOS + Android from a single codebase)
- Backend: Firebase Auth, Firestore, Cloud Functions (Node.js)
- APIs: TMDB (movie catalog), OMDb (external ratings - IMDb, Rotten Tomatoes, Metacritic)
- ML Persistence: SharedPreferences (local, cross-session ML state)
- Architecture: Provider pattern (6 ChangeNotifiers), singleton services, 64-dimensional vector embeddings
- AI/ML: Embeddings, Collaborative Filtering, Matrix Factorization, Online Learning, Adaptive Weighting, Behavioral Tracking, Contextual Bandits

2. Core AI Architecture - The Orchestrator Pattern

The recommendation engine is a hybrid orchestrated system with 7 concurrent AI/ML services - architecturally identical to how modern AI agent systems work. The central method `MovieProvider.loadPersonalizedRecommendations()` acts as the ORCHESTRATOR (equivalent to an AI agent runner), coordinating specialized modules like tools.

Stage 1 - Retrieval (4 Parallel Discovery Strategies)

- Trending movies from TMDB (real-time popularity signal)
- Genre-based discovery filtered by the user's top 3 preferred genres
- Content similarity from the user's top-8 liked movies (`getSimilarMovies` + `getMovieRecommendations`)
- Actor/director-based discovery from the user's top 5 actors and top 3 directors

Stage 2 - Scoring (`_scoreMovies`)

- Base Score (50%): genre match (40%), actor match (25%), director match (20%), rating (10%), recency (5%)
- `MovieEmbeddingService` (15%): 64-dimensional vector cosine similarity
- `ContextualRecommendationService` (15%): time of day, mood, weekday/weekend
- `BehaviorTrackingService` (10%): swipe speed, detail view duration, revisit count
- `CollaborativeFilteringService` (5%): co-occurrence matrix with popularity normalization
- `DeepLearningService` (5%): 18-dimensional feature vector with rule-based fallback (TFLite infrastructure ready)

Stage 3 - Filtering & Deduplication

- Remove already-liked, disliked, watchlisted, and skipped items
- Platform availability filter via `StreamingService` (TMDB watch/providers API)
- Diversity filter: prevents genre clustering in the visible deck (max 2 genre overlaps in first 10 cards)

Stage 4 - Adaptive Meta-Learning

- `AdaptiveWeightingService` learns which of the 5 strategies works best per individual user using exponential smoothing
- `MatrixFactorizationService` maintains 20-dimensional user and movie embedding vectors updated via gradient descent
- `OnlineLearningService` triggers incremental embedding updates after each interaction (rate-limited to 5-second intervals)

3. RAG - What It Is and How PopMatch Implements the Pattern

What is RAG?

Retrieval-Augmented Generation (RAG) is an AI architecture pattern that solves the core limitations of pure LLMs (stale knowledge, hallucination, missing domain data). Instead of answering from training weights alone, the system:

- RETRIEVES relevant documents/records from an external knowledge base at query time
- AUGMENTS the model's context with that retrieved, up-to-date content
- GENERATES a response grounded in the retrieved information

PopMatch as a RAG System (Without the LLM)

PopMatch's recommendation pipeline is structurally identical to a RAG system:

RAG Component	PopMatch Equivalent
Knowledge Base	TMDB movie catalog + OMDb ratings + social activity feed
Retrieval	4 parallel TMDB query strategies (Stage 1)
Augmentation	User behavioral context injected into scoring (liked movies, swipe speed, time of day)
Generation	_scoreMovies() produces a personalized ranked list
Query	User's implicit query = like/dislike history + mood + current time

Real LLM-Based RAG System (Production)

When asked about production RAG with LLMs, the steps are:

- Embed user query + documents into the same vector space using a pre-trained model (e.g., text-embedding-3-small, bge-m3, e5-mistral-7b)
- Store document embeddings in a vector DB (Pinecone, Weaviate, Chroma, pgvector)
- At query time: embed the query, run ANN search (FAISS / HNSW) to retrieve top-K docs in milliseconds
- Inject retrieved docs into the LLM prompt as grounding context
- Generate response - grounded, not hallucinated

PopMatch's MovieEmbeddingService implements steps 1-3 manually in Dart - the same mathematical principles, just without an LLM at the end. Building it from scratch proves understanding of the underlying math, not just API calls.

4. Embeddings - Deep Technical Explanation

What Are Embeddings?

An embedding is a dense, fixed-size numerical vector that encodes the semantic meaning of a data point. The fundamental property: semantically similar items have vectors that are geometrically close in the embedding space, measured by cosine similarity.

```
cosine_similarity(A, B) = (A . B) / (||A|| x ||B||)
Range: [-1, 1], where 1 = identical, 0 = orthogonal, -1 = opposite
```

PopMatch's 64-Dimensional Movie Embedding Vector (MovieEmbeddingService)

Dimensions	Count	Feature	Encoding
0 - 19	20	Genre presence	One-hot per genre (action=1, comedy=0, etc.)
20 - 24	5	Vote average	rating / 10.0, repeated 5x for weight
25 - 29	5	Release year	(year - 1900) / 130
30 - 34	5	Popularity	popularity / 100, clamped to 1.0

PopMatch - AI/ML Interview Preparation Guide

35 - 39	5	Runtime	runtime / 200
40 - 44	5	Vote count	log(voteCount + 1) / 15
45 - 49	5	Language	hashCode % 100 / 100 (hash feature)
50 - 55	6	Text keyword freq.	Genre keywords in overview text (NLP)
56	1	Description length	wordCount / 200
57	1	Vocabulary diversity	unique words / total words
58	1	Positive emotion	(happy, joy, win, hero) / total words
59	1	Negative emotion	(death, dark, fear, evil) / total words
60 - 63	4	Thematic keywords	space, future, past, journey prevalence

After construction: L2-normalize the vector so all vectors have unit length. Cosine similarity then reduces to a simple dot product.

Dual Similarity Scoring

```
for each likedMovie in user.likedMovies:
    sim = cosine_similarity(candidate_vec, liked_vec)

final = 0.6 x max_similarity + 0.4 x avg_similarity
weight = 0.3 + (final x 0.7) # scaled to [0.3, 1.0]
```

max captures 'does this exactly match something the user loves?' - avg captures 'does this consistently fit their overall taste profile?' The dual approach handles both passionate matches and consistent preferences.

Production Scale

- Pre-trained embedding models: text-embedding-ada-002, bge-m3, e5-mistral-7b
- Vector databases: Pinecone, Weaviate, Qdrant, pgvector
- ANN search: FAISS (Facebook), HNSW (Hierarchical Navigable Small World) - billions of vectors in milliseconds
- Building embeddings from scratch (as in PopMatch) proves understanding of the math, not just API usage

5. Machine Learning Training - Six Techniques

5.1 Item-Based Collaborative Filtering (CollaborativeFilteringService)

Every addLikedMovie() call is a real-time training step. The model updates a co-occurrence matrix - a form of online learning, not batch training.

```
For every user who liked movie A AND movie B:
    co_occurrence[A][B] += 1

Score for candidate C given user liked A:
    raw_score = co_occurrence[A][C]
    normalized = raw_score / sqrt(movie_popularity[C])
    # sqrt dampens blockbuster bias - surfaces niche relevant films
```

Square-root popularity normalization prevents popular blockbusters (Avengers, Inception) from dominating co-occurrence counts, giving niche but relevant films a fair chance.

5.2 Matrix Factorization - SVD-Like Decomposition (MatrixFactorizationService)

Implements a Singular Value Decomposition (SVD)-style approach with 20 latent factors. This is the same technique that powered the Netflix Prize winning algorithm.

```
# Latent factors: 20 dimensions
# Learning rate: 0.01 Regularization: 0.1
```

```
# User embedding: userId -> List<double> (20D vector)
# Movie embedding: movieId -> List<double> (20D vector)

# Prediction:
predicted_rating = dot(user_vec, movie_vec) # normalized to [0, 1]

# Gradient descent update:
error = actual_rating - predicted_rating
user_vec += lr * (error * movie_vec - reg * user_vec)
movie_vec += lr * (error * user_vec - reg * movie_vec)
```

Key methods: predict(userId, movieId), updateFromFeedback(), getMovieSimilarity(), getUserSimilarity(), getRecommendedMovies(). Embeddings are persisted to SharedPreferences as JSON.

5.3 Adaptive Meta-Learning (AdaptiveWeightingService)

The system does not just score movies - it learns which scoring strategies work best for each individual user and adjusts weights continuously via exponential smoothing. This is a multi-armed bandit problem.

```
Initial weights per strategy:
contentBased = 40%, contextual = 20%, embedding = 20%
behavior = 15%, collaborative = 5%

After >= 10 feedbacks per strategy:
success_rate = likes / (likes + dislikes)
# Exponential moving average:
new_weight = (old_weight * 0.7) + (target_weight * 0.3)
# Re-normalize: all weights must sum to 1.0
```

Context-aware cold/warm start adjustment:

- New users (<5 likes): boost contentBased x1.2 + contextual x1.2, reduce embedding x0.8
- Experienced users (>20 likes): boost embedding x1.3 + collaborative x1.5, reduce contentBased x0.9
- Active session: boost behavior weight x1.2

5.4 Online Learning (OnlineLearningService)

Incremental learning from every user interaction (like, dislike, skip, view). Rate-limited to 5-second minimum intervals to avoid excessive updates. Ratings: like=+1, dislike=-0.5, neutral=0. Triggers MatrixFactorization gradient descent updates and updates embeddings incrementally when preferences change. Stores update history in SharedPreferences for model freshness tracking.

5.5 Implicit Behavioral Feedback (BehaviorTrackingService)

Converts interaction patterns into continuous interest scores - the same technique used by Netflix and Spotify. Implicit feedback is often more predictive than explicit ratings.

```
Interest score for a movie =
(detailViewCount * 0.3)
+ clamp(avgTimeOnDetailPage / 10.0, 0, 2.0)
+ (revisitCount * 0.5)
+ 0.2 if swipeSpeed > 2000ms # hesitation = interest
+ explicitSignal # +1.0 like, -0.5 dislike

behavior_weight = 0.5 + (normalized_interest * 0.5) # [0.5, 1.0]
behavior_weight = 0.5 - (normalized_disinterest * 0.5) # [0.0, 0.5]
```

5.6 Contextual Multi-Armed Bandits (ContextualRecommendationService)

Adjusts scoring weights based on observable context - time of day, day of week, and current mood. The reward function changes with context rather than being stationary.

```
Morning (6-12): comedy x1.2, animation x1.15, horror x0.8
Evening (17-22): action x1.15, drama x1.1, thriller x1.1
Night (22-6): horror x1.2, thriller x1.15, animation x0.85
```

```
Weekend: prefer longer movies (>120 min) x1.1
Weekday: prefer shorter movies (<100 min) x1.05

Mood multiplier: 1.0 + (matchingGenres * 0.15)
```

5.7 Deep Learning Infrastructure (DeepLearningService)

TensorFlow Lite infrastructure deployed, model not yet trained. Uses rule-based fallback scoring. Feature vector ready for neural network training.

```
18-dimensional feature vector:
[0] user.likedMovieCount (normalized)
[1] user.dislikedMovieCount (normalized)
[2] movie.voteAverage / 10.0
[3] movie.popularity / 100.0
[4] movie.voteCount / 1000.0
[5-14] genre one-hot (top 10 TMDB genre IDs)
[15] (releaseYear - 1900) / 130.0
[16] runtime / 200.0
[17] 0.0 (reserved)

Next step: collect labeled swipe data (right=1, left=0),
train 3-4 layer feedforward network, export to .tflite
```

6. AI Agents - Framing the Architecture

What Is an AI Agent?

An AI agent is a system that: Perceives its environment (input) -> Reasons about what to do (planning/decision) -> Acts (calls tools/APIs) -> Observes the outcome -> Learns and updates its state.

PopMatch's Recommendation System as an AI Agent

Agent Property	PopMatch Implementation
Perception	User swipes, view times, time of day, liked movie history, current mood
Reasoning	_scoreMovies() - multi-factor weighted evaluation of 100+ candidates
Tool Use	TMDB API, OMDb API, StreamingService - external tools called dynamically
Action	Surfaces ranked movie list to the swipe interface
Observation	Records outcome: liked? disliked? opened detail? time spent?
Learning	AdaptiveWeightingService + OnlineLearningService update models from outcomes
Short-term Memory	BehaviorTrackingService in-memory session signals
Long-term Memory	SharedPreferences: co-occurrence matrix, MF embeddings, strategy weights

Production LLM-Based Agent (Like Microsoft Copilot)

The same pattern implemented with LLMs using the ReAct (Reason + Act) loop:

```
# ReAct agent loop
while not done:
    thought = llm.think(system_prompt, history, current_observation)
    action = llm.choose_tool(thought, available_tools)
    observation = tool_registry[action.name].run(action.params)
    history.append((thought, action, observation))
    done = llm.should_stop(history)
```

Key LLM agent concepts: Tool use / Function calling, Chain-of-thought planning, Short-term memory (conversation context) +

Long-term memory (vector DB), Orchestration frameworks: LangChain, LlamaIndex, Microsoft Semantic Kernel (for Copilot), CrewAI, AutoGen.

7. Data Architecture - Full Pipeline

User Model - What Is Stored

The User model carries all preference and interaction state used to drive personalization:

- Authentication: id (Firebase UID), email, displayName, photoURL
- Movie tracking: likedMovies, dislikedMovies, watchlist - the primary training signals
- Show tracking: likedShows, dislikedShows, watchlistShows
- Context: currentMood (string) - drives ContextualRecommendationService
- Preferences map: onboardingCompleted (bool), selectedGenres (List<int>), selectedPlatforms (List<String>)

Onboarding -> Recommendation Integration

The 3-step onboarding (welcome -> genre selection -> platform selection) seeds the cold-start recommendation engine:

- Genres selected during onboarding become the initial preference signal
- UserPreferenceAnalyzer uses onboarding genres as defaults when liked movie count is below threshold
- Blend weight (0.0 -> 1.0) progressively personalizes recommendations as likes accumulate (0 likes = pure onboarding, 3+ likes = full personalization)
- UserPreferenceAnalyzer extracts top genres, actors, directors, and rating ranges from up to 30 liked movies in parallel
- Rating range: 25th/75th percentile of liked movie ratings (ignores outliers) - defines the user's quality bar

Performance Optimizations

- Infinite swipe buffer: 30-movie minimum pre-loaded before they are needed - zero wait on swipe
- Staged loading: instant load from disk cache -> fresh load in background -> merge seamlessly
- _pendingMovies queue: background-loaded movies held until main deck empties - no mid-swipe pop-in
- Session cache: UserPreferencesSessionCache avoids reanalyzing user preferences on every tab switch
- MovieCacheService: 24-hour TTL, 50-movie LRU cache for detail screens - preloads before navigation
- Rate limiting: 850ms-900ms throttle on preload kicks to avoid API flooding
- Deferred removal: card removed from deck 4 seconds after swipe - allows undo without flickering

Complete Data Flow

```

TMDB API
-> TMDBService (8s timeout, test mode support)
-> Movie.fromJson() deserialization
-> MovieProvider._scoreMovies()
    |- UserPreferencesSessionCache (top genres, actors, directors)
    |- Base scoring (genre/actor/director/rating/recency)
    |- MovieEmbeddingService (64D cosine similarity)
    |- ContextualRecommendationService (time/mood)
    |- BehaviorTrackingService (implicit signals)
    |- CollaborativeFilteringService (co-occurrence)
    |- MatrixFactorizationService (SVD-like)
-> _applyDiversityFilter()
-> StreamingService platform filter
-> _filteredMovies (visible swipe deck)
-> RetroCinemaMovieCard (UI)
-> User swipes
-> AuthProvider (like/dislike/watchlist recorded)
-> OnlineLearningService (incremental MF update)
-> BehaviorTrackingService (implicit signal update)
-> AdaptiveWeightingService (strategy weight update)
-> OMDb API (background: IMDb/RT/Metacritic ratings)
  
```

8. Social Architecture & Governance

Social Features (SocialService + Cloud Functions)

Social features run entirely through Firebase Cloud Functions (Node.js, region: us-central1) with Firestore as the data store. The client-side SocialService gracefully degrades when Firebase is disabled.

- Follow system: sendFollowRequest(), respondToFollowRequest(), unfollow()
- User discovery: searchUsers() (case-insensitive, by name/email), getSuggestedUsers() (Jaccard similarity - minimum 2 shared liked movies)
- Social activity feed: recordActivity() respects per-user privacy settings server-side before writing to Firestore
- Friends feed: getFriendsFeed() filters out activities marked private - privacy enforced at Cloud Function level, not client

Governance Layers

Governance in PopMatch runs at three levels:

- Algorithm level: A/B testing with 3 persistent variants (variantA 50%, variantB 30%, variantC 20%). Tracks Precision@K, Recall@K, NDCG@K, Diversity, Novelty, and Coverage per variant
- Privacy level: Social activity sharing enforced server-side in Cloud Functions - client cannot bypass visibility rules
- Observability level: Every recommendation carries a recommendationStrategy field identifying which algorithm surfaced it
- explainability built in
- Feature flags: Compile-time dart-define flags (SOCIAL_UI_ENABLED, FRIENDS_FEED_ENABLED) for controlled feature rollout

Recommendation Metrics (RecommendationMetricsService)

Metric	Formula	Weight
Precision@K	Liked items in top K / K	30%
Recall@K	Liked items in top K / total liked	25%
NDCG@K	DCG / IDCG (log-discounted rank quality)	25%
Diversity	Genre spread across recommendations	10%
Novelty	Inverse popularity (surfacing non-obvious items)	5%
Coverage	Fraction of catalog that can be recommended	5%

Overall quality score = Precision(0.30) + Recall(0.25) + NDCG(0.25) + Diversity(0.10) + Novelty(0.05) + Coverage(0.05). Stored in SharedPreferences (last 1000 evaluations) with timestamps for temporal analysis.

9. Python Backend Mapping

The job requires Python backend development. Every PopMatch component maps directly to Python equivalents:

PopMatch (Dart)	Python Equivalent
TMDBService	FastAPI + httpx async client
MovieEmbeddingService	numpy + sklearn.preprocessing.normalize + cosine_similarity
CollaborativeFilteringService	scipy.sparse lil_matrix
MatrixFactorizationService	numpy gradient descent or surprise library (SVD)
Future.wait() parallel calls	asyncio.gather() or ThreadPoolExecutor
SharedPreferences ML state	Redis (with TTL) or PostgreSQL
Background preload timer	Celery async tasks + Redis broker
AdaptiveWeightingService	scipy.stats + scikit-learn
A/B significance test	scipy.stats.ttest_ind (p_value < 0.05)
Vector DB (production)	Pinecone / pgvector / Weaviate
Cloud Functions (Node.js)	FastAPI microservices or AWS Lambda (Python)

Python Recommendation API Example

```
@app.post('/recommendations')
async def get_recommendations(user: UserProfile, limit: int = 30):
    # Stage 1: Parallel retrieval
    candidates = await asyncio.gather(
        tmdb.get_trending(),
        tmdb.discover_by_genres(user.top_genres),
        tmdb.get_similar(user.top_liked_ids[:8])
    )
    all_candidates = deduplicate(flatten(candidates))

    # Stage 2: Multi-factor scoring
    for movie in all_candidates:
        base = compute_base_score(movie, user)      # 50%
        embed = embedding_svc.get_weight(movie, user) # 15%
        ctx = contextual_svc.get_weight(movie)       # 15%
        behav = behavior_svc.get_weight(movie.id)    # 10%
        collab = cf_svc.get_score(movie.id, user)    # 5%
        score = 0.50*base + 0.15*embed + 0.15*ctx \
            + 0.10*behav + 0.05*collab

    # Stage 3: Diversity filter + return
    return diversity_filter(sorted(scored, reverse=True))[:limit]
```

Python Embedding Service Example

```
import numpy as np
from sklearn.preprocessing import normalize
from sklearn.metrics.pairwise import cosine_similarity

def build_embedding(movie: Movie) -> np.ndarray:
    vec = np.zeros(64)
    for i, gid in enumerate(movie.genre_ids[:20]):
        if gid in GENRE_INDEX: vec[GENRE_INDEX[gid]] = 1.0
    vec[20:25] = movie.vote_average / 10.0
    vec[25:30] = (movie.release_year - 1900) / 130.0
    # ... remaining dimensions
    return normalize(vec.reshape(1, -1))[0] # L2 normalize

def get_weight(candidate, liked_movies) -> float:
    c_vec = build_embedding(candidate)
    l_vecs = np.array([build_embedding(m) for m in liked_movies])
    sims = cosine_similarity([c_vec], l_vecs)[0]
    return 0.6 * sims.max() + 0.4 * sims.mean()
```

10. LLM Orchestration, Performance, and Governance

Orchestration Patterns

- LangChain / LlamaIndex: Pipeline-based orchestration for RAG + agents
- Microsoft Semantic Kernel: Copilot-specific framework - Planner + Skills + Memory + Connectors
- CrewAI / AutoGen: Multi-agent systems where specialized sub-agents collaborate
- Prefect / Airflow: Orchestrating ML pipelines and data workflows in production

Performance Patterns (from PopMatch)

- Parallelism: 4 discovery strategies run simultaneously (Future.wait in Dart, asyncio.gather in Python)
- Fallback chains: personalized fails -> curated starter movies. In production: define retry/fallback at the orchestration layer
- Rate limiting: 850ms throttle on preload kicks. In production: token bucket or leaky bucket algorithm
- Multi-level caching: session cache (in-memory) + disk cache (SharedPreferences) + TTL-based expiry

Production AI Governance (Beyond PopMatch)

- Model cards: document training data, evaluation metrics, known failure modes, intended use

- Bias detection: monitor recommendation distribution across demographic groups
- Explainability: the recommendationStrategy field (already in PopMatch's Movie model) explains why each item was surfaced
- PII handling: never store raw behavioral data with user PII in the same store
- Drift detection: compare current recommendation distributions against baseline - alert on significant shifts
- Human-in-the-loop: escalation paths for low-confidence agent decisions

11. Interview Talking Points

On AI Agents

"In PopMatch, I built a recommendation agent that follows the classic perceive-reason-act-learn loop. The orchestrator coordinates 7 specialized modules, each providing a different signal. This is architecturally identical to tool-using LLM agents - the agent decides which tools to call (TMDB discover, OMDb ratings, streaming availability), collects results, and synthesizes a ranked output. In a Copilot-based system I would add natural language planning using Semantic Kernel's Planner before tool invocation."

On RAGs

"PopMatch implements the retrieval-augment-generate pattern without an LLM. The retrieval stage runs 4 parallel TMDB queries. The augmentation injects behavioral context - liked movies, swipe speed, time of day. The generation is the scoring function that produces a ranked list. In an LLM-based RAG, you replace the scoring function with a language model reasoning over retrieved documents. I understand both the indexing side (building embeddings, storing in vector DBs) and the query side (ANN search, re-ranking, context window management)."

On Embeddings

"I hand-built 64-dimensional movie embeddings encoding genre, rating, year, popularity, runtime, and NLP text features from movie descriptions. I implemented L2 normalization and cosine similarity from scratch with a dual max+average approach that captures both passionate matches and consistent taste profiles. In production I would use a pre-trained model and a vector DB, but building it manually means I fully understand the underlying math."

On ML Training

"I implemented five forms of ML training: (1) online collaborative filtering - every like updates a co-occurrence matrix in real-time; (2) matrix factorization - SVD-like 20-dimensional latent factor model updated via gradient descent; (3) adaptive meta-learning - exponential smoothing learns optimal strategy weights per user; (4) implicit feedback - swipe speed, detail page duration, and revisit count become continuous interest scores; (5) contextual bandits - genre weights shift based on time of day and mood."

On Governance

"Governance runs at three levels: (1) algorithm - A/B testing 3 variants tracking precision, recall, NDCG, diversity, novelty, and coverage; (2) privacy - social activity visibility enforced server-side in Cloud Functions, clients cannot bypass; (3) observability - every recommendation carries a recommendationStrategy field for explainability. In production I would add model cards, drift detection, and bias auditing."

On Python Backend

"All ML logic in PopMatch is in Dart. I can map every component to Python: FastAPI for the recommendation API, httpx for async TMDB calls, numpy/sklearn for embeddings, scipy for collaborative filtering, Redis for caching, Celery for background pre-computation, and pgvector or Pinecone for embedding storage. The architecture is identical - only the language changes."

12. Common Interview Questions & Answers

Q: What is RAG and when would you use it?

Retrieval-Augmented Generation retrieves relevant documents from an external knowledge base at query time to ground LLM responses. Use it when: model training data is stale, domain-specific knowledge is required, or hallucination is unacceptable.

Requires: embedding model + vector DB + re-ranker + LLM. PopMatch is structurally a RAG system - the scoring function plays the LLM's role.

Q: How do embeddings work?

Dense numerical vectors where similar items cluster together in vector space. Built by encoding semantic features into fixed-size arrays and L2-normalizing. Similarity measured by cosine distance (dot product after normalization). In PopMatch: 64D manual embeddings. In production: pre-trained models (ada-002, bge-m3) stored in Pinecone/Weaviate, searched with FAISS/HNSW.

Q: Collaborative vs content-based filtering?

Content-based: compare item attributes (genres, cast, description) - 'this movie shares your liked genres.' Collaborative: compare user behavior - 'users who liked what you liked also liked this.' PopMatch implements both and combines them with adaptive weights. Cold-start problem: collaborative needs data, content-based works immediately.

Q: How do you handle cold start?

Below 3 likes: curated popular + onboarding genre-based recommendations (content-based only, no collaborative signal). Above 3 likes: blend weight increases progressively to 1.0 - full personalized pipeline with all 7 services. For LLM agents: onboarding prompts build an explicit preference prior before behavioral data exists.

Q: What is NDCG?

Normalized Discounted Cumulative Gain - measures ranking quality. Items at position 1 contribute more than position 10 (logarithmic discount: $1/\log_2(\text{rank}+1)$). Normalized means relative to the ideal ranking (IDCG). Formula: $\text{NDCG@K} = \text{DCG@K} / \text{IDCG@K}$. Used in PopMatch's RecommendationMetricsService with 25% weight in overall quality score.

Q: What is matrix factorization?

Decomposes the user-item interaction matrix into two low-dimensional matrices: user embeddings and item embeddings. The dot product of a user vector and item vector predicts the user's rating. SVD (Singular Value Decomposition) is the canonical approach - powered the Netflix Prize winning algorithm. PopMatch implements this with 20 latent factors, learning rate 0.01, and L2 regularization 0.1.

Q: What is a multi-armed bandit?

An online learning problem where an agent chooses between K strategies (arms), observes a reward, and learns which arms yield the highest reward over time. In PopMatch: each of the 5 scoring strategies is an arm. AdaptiveWeightingService is the bandit - it uses exponential smoothing (not pure exploration/exploitation) to converge on optimal per-user weights.

Q: How would you scale this to production in Python?

FastAPI microservice for the recommendation API, async httpx for TMDB, Redis for caching embeddings and scores (TTL 24h), Celery for background pre-computation triggered by user events, PostgreSQL + pgvector for storing movie embeddings, Pinecone or Weaviate for ANN search at scale, Prometheus + Grafana for metrics, Optimizely or custom for A/B test management, MLflow for model tracking and versioning.

13. Critical Files to Review

File	Why It Matters
lib/services/movie_embedding_service.dart	Core embeddings - know all 64 dimensions and dual scoring
lib/services/collaborative_filtering_service.dart	Co-occurrence matrix, sqrt popularity normalization
lib/services/matrix_factorization_service.dart	SVD-like 20D latent factors, gradient descent updates
lib/services/online_learning_service.dart	Incremental learning loop, rate limiting
lib/services/adaptive_weighting_service.dart	Meta-learning, exponential smoothing, multi-armed bandit
lib/services/behavior_tracking_service.dart	Implicit feedback signals, interest score formula

PopMatch - AI/ML Interview Preparation Guide

lib/services/contextual_recommendation_service.dart	Circadian patterns, mood multipliers
lib/providers/movie_provider.dart	Full orchestration pipeline - the agent loop
lib/services/ab_testing_service.dart	Variant assignment, metrics, statistical comparison
lib/services/recommendation_metrics_service.dart	Precision, Recall, NDCG formulas
lib/services/user_preference_analyzer.dart	Preference extraction, blend weight, cold start
functions/index.js	Cloud Functions, social governance, privacy enforcement
docs/RECOMMENDATION_ALGORITHM.md	Exact weight breakdown for the full scoring pipeline